

BINARY SEARCH TREE

(一) 程式簡介

本程式使用 C++ 模擬二元搜尋樹的一系列操作。使用者輸入 4 行數值，並輸出 10 行的結果。

範例如下：

輸入	意義
5	要放入二元樹中的數值數量。
3 6 1 4 5	要放入二元樹中的數值。
2	X 值。
3	Y 值。
輸出	意義
1 5 4 6 3	以後序表示法印出含有 5 個數值的二元樹。
5 4 6 1 3	以後序表示法印出二元樹的鏡像。
3 1 2 6 4 5	將 X 值放入二元樹中，再以前序表示法印出。
4	二元樹的高度。
6	二元樹的節點數量。
4	二元樹的內部節點數量。
6	二元樹中的最大值。
1	二元樹中的最小值。
2 1 6 4 5	刪除二元樹中的 Y 值，並以左子樹中的最大值替代後，用後序表示法印出。
4 1 2 6 5	刪除二元樹中的 Y 值，並以右子樹中的最小值替代後，用後序表示法印出。

(二) 執行畫面

```

Microsoft Windows [版本 10.0.17134.345]
(c) 2018 Microsoft Corporation. 著作權所有，並保留一切權利。
D:\Aya>D:\Aya\B10632032 HW2.exe
5
3 6 1 4 5
2
3
1 5 4 6 3
5 4 6 1 3
3 1 2 6 4 5
4
6
4
6
1
2 1 6 4 5
4 1 2 6 5
  
```

(三) 介紹型別: Node

本程式的 Node 型別包含兩格 Node 指標和一格 data, 藉由 left 和 right 指向左子樹和右子樹。



(四) 介紹函式

- ① void insert(Node** tree, int value)

用途: 將數值放入二元樹中。

做法: 當*tree 為 NULL, 使用 malloc 配置新節點的空間, 將數值放入, 並將 left 和 right 初始為 NULL; 若*tree 不為 NULL, 且新數值大於*tree 的值, 則將新數值放入右子樹中, 若新數值小於*tree 的值則放入左子樹。

- ② void clean(Node** tree)

用途: 將二元樹全部刪除, 歸還記憶體。

做法: 以遞迴方式由左右子樹往根節點 free 整個二元樹。

- ③ void preorder(Node* tree) 與 void postorder(Node* tree)

用途: 以前序表示法 / 後序表示法印出二元樹。

做法: 前序表示法是以遞迴方式先印根節點, 再印左子樹, 最後印右子樹; 後序表示法則是先印左子樹, 再印右子樹, 最後印根節點。

- ④ void mirror(Node** tree)

用途: 將二元樹變成自己的鏡像。

做法: 以遞迴方式從葉節點開始交換 left 和 right。

- ⑤ int height(Node* tree)

用途: 計算二元樹的高度。

做法: 以遞迴方式從葉節點開始找到高度較高的子樹, 往根節點累計, 算出根節點與葉節點距離的最大值。

- ⑥ int total(Node* tree) 與 int external(Node* tree)

用途: 計算二元樹的節點 / 葉節點數量。

做法: 以遞迴方式往下找到左右皆為 NULL 的葉節點, 並加總左右子樹的所有節點或葉節點數量。

- ⑦ Node* max(Node* tree) 與 Node* min(Node* tree)

用途: 找出二元樹中含有最大值 / 最小值的節點。

做法: 最大值的部分, 只要 right 不為 NULL, 就將指標往右子樹移動, 直到遇到 right 為 NULL。最小值則是往左子樹移動。

- ⑧ void del_l (Node** tree, int value) 與 void del_r (Node** tree, int value)

用途: 刪除二元樹中的某個數值。

做法: 當*tree 的值並非目標, 視 value 的大小往 left 或 right 移動; 而當遇到要刪除的值時, 分為三種狀況: 1.沒有子節點就直接 free 掉目標, 並將指標改為 NULL; 2.有一個子節點, 就以子節點替代目標的位置; 3.有兩個子節點則用左子樹的最大值 / 右子樹的最小值代替。

(五) 心得

在寫指標相關的程式時，我可能還是不夠熟悉，經常會指錯地方，要除錯就必須印出很多資訊，才能慢慢找出問題點。我是一次寫一個 function，確認沒問題才繼續往下寫，感覺這樣比較不會一次遇到一堆問題，很難找；但還是花了不少時間。

這次用解題系統，我一直有幾筆測資不過，又找不到原因；後來才發現我的 insert 其實忘記把 left 跟 right 初始為 NULL，但是因為我用線上的 compiler 並沒有報錯，所以我一直以為是 del 的問題，還寫了產生測資的程式試了好幾天。看來寫程式還是好好用個 IDE 比較好。

另外我發現遞迴的方式真的很特別，有些本來以為需要寫進去的變數或結構，其實用遞迴的方式就變得很簡潔。例如我直覺覺得要刪除一些節點，可能要先 search 到那個節點，然後 Node 還得包含指向 parent 的指標，才能知道是誰要指向替代的節點等等。想了很複雜的各種 if else，指標一直移動，後來看了講義上遞迴的方式，發現簡單很多。